OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:
Opticrop is an intelligent, data-driven system designed to revolutionize modern agriculture by leveraging the power of IoT, data analytics, and machine learning. Its primary objective is to assist farmers in making informed, real-time decisions that enhance crop yield, conserve resources, and support sustainable farming practices.

Through a seamless integration of smart sensors, predictive analytics, and intuitive user interfaces, Opticrop continuously monitors critical environmental and crop-related parameters. These insights enable precision farming strategies tailored to the specific needs of each farm—ensuring maximum productivity with minimal input waste.

The system is engineered to tackle real-world agricultural challenges such as unpredictable weather, pest invasions, inconsistent irrigation, and poor crop health visibility—ultimately transforming the way farmers manage their fields.

Key Features:

- Real-time Data Collection: Using IoT sensors to monitor soil moisture, temperature, humidity, rainfall, and light levels.
- Predictive Analytics: Forecast irrigation needs, detect early pest activity, and predict crop health issues using ML models.
- Automated Recommendations: Suggest optimal actions for watering, fertilization, pesticide use, and harvesting.
- User-Friendly Dashboard: Visualize data trends, crop conditions, and actionable insights in an intuitive web interface.
- Sustainability Focused: Helps in minimizing resource waste, reducing chemical usage, and promoting eco-friendly farming.

Scenarios

Scenario 1: Smart Crop Recommendation for Farmers

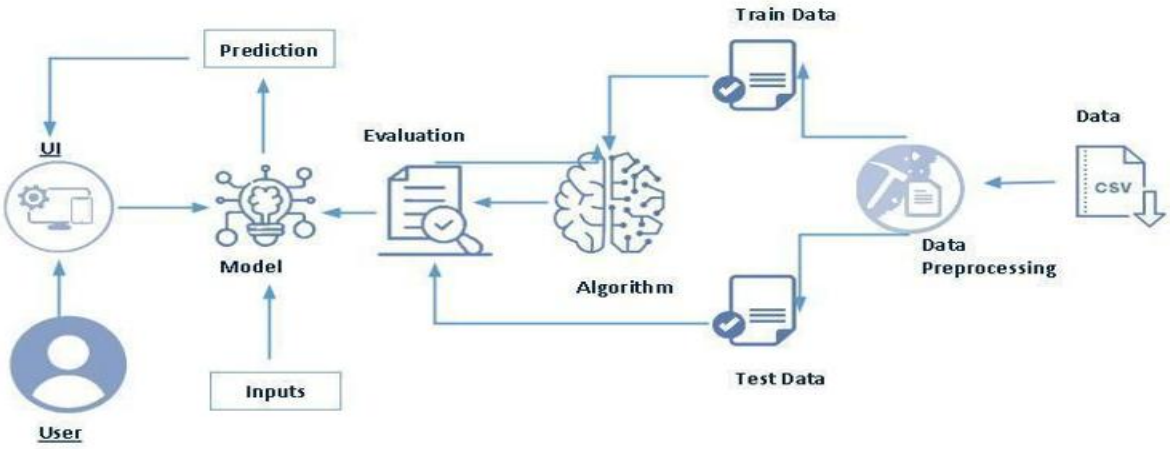
A farmer enters soil nutrient values such as Nitrogen, Phosphorous, Potassium, along with environmental conditions including temperature, humidity, pH level, and rainfall. The system processes these parameters and recommends the most suitable crop for achieving maximum yield and efficient resource utilization.

Scenario 2: Crop Suitability and Environmental Assessment
A user wishes to determine whether current soil and climatic conditions are appropriate for cultivating a specific crop. The system evaluates environmental compatibility and provides insights into crop suitability, productivity potential, and agricultural feasibility.

Scenario 3: Agricultural Research and Policy Planning
Agricultural researchers and policymakers utilize the platform to study relationships between crop

production and environmental factors. The system supports evidence-based decision-making for sustainable agriculture, resource management, and strategic agricultural planning.

Technical Architecture:



Description:

OptiCrop follows a machine learning-based architecture consisting of data collection, data preprocessing, exploratory data analysis, model training, prediction, and web application deployment.

The system architecture includes:
1.Frontend developed using HTML, CSS, and JavaScript.
2.Backend developed using Flask.
3.Machine learning models implemented using Scikit-learn.
4.Data visualization using Matplotlib and Seaborn.
5.Model deployment using Flask web framework.

The workflow of the system is:
User Input → Data Processing → Machine Learning Model → Crop Prediction → Result Display.

Pre-requisites:

- Python Programming Knowledge
- Machine Learning Fundamentals
- NumPy and Pandas Libraries
- Scikit-learn Framework
- Flask Web Framework
- Data Visualization using Matplotlib and Seaborn
- Jupyter Notebook/Spyder
- Git Version Control
- Visual Studio Code or PyCharm IDE

Project Workflow:

Milestone 1: Data Collection and Preprocessing

Activity 1.1: Dataset Collection

The Crop Recommendation dataset was collected from Kaggle and imported into the project environment. The dataset contains soil nutrient values, climatic parameters, and crop labels.

Activity 1.2: Data Preprocessing

The dataset was cleaned and analyzed by:

Checking missing values

Detecting outliers

Handling feature distributions

Splitting data into training and testing datasets

Milestone 2: Data Visualization and Analysis

Activity 2.1: Crop Distribution Analysis

The crop distribution chart was generated to visualize the frequency distribution of crops present in the dataset.

Description: The crop distribution graph demonstrates that the dataset contains balanced crop classes, ensuring effective model training and reducing prediction bias.

Activity 2.2: Correlation Analysis

A correlation heatmap was generated to understand relationships among agricultural parameters.

Description: The heatmap illustrates positive and negative relationships among Nitrogen, Phosphorus, Potassium, temperature, humidity, pH, and rainfall variables, helping identify feature dependencies.

Activity 2.3: Feature Distribution Analysis

Histograms were generated to analyze the distribution of all numerical features.

Description: Histogram plots provide insights into data spread, skewness, and concentration patterns across agricultural features, facilitating feature engineering and preprocessing decisions.

Milestone 3: Model Building

Activity 3.1: Machine Learning Model Development

Several machine learning algorithms were evaluated for crop prediction:

Logistic Regression

Random Forest

Decision Tree

K-Means Clustering

The models were trained using the processed dataset, and performance metrics were evaluated to determine the best-performing model.

Activity 3.2: Model Evaluation

The trained models were evaluated using:

Accuracy Score

Confusion Matrix

Classification Report

Precision

Recall

F1 Score

The best-performing model was saved using pickle/joblib for deployment.

Milestone 4: Application Development

Activity 4.1: Frontend Development

The frontend was developed using HTML, CSS, and JavaScript to provide an interactive user interface for entering agricultural parameters and viewing crop predictions.

Activity 4.2: Backend Development

The backend was developed using Flask to:

Accept user inputs.

Process agricultural parameters.

Load the trained model.

Generate crop predictions.

Display the recommendation results.

Milestone 5: Application Deployment

Activity 5.1: Running the Application

The Flask application was executed using Visual Studio Code. Users can access the system through a local web browser and obtain crop recommendations by entering soil and environmental parameters.

Project Structure:

Create the OPTICROP project directory with the following structure:

Data Collection & Data Pre-processing

Download the dataset

To begin the project, the required dataset was sourced from a publicly available open-source platform. Numerous such repositories exist, including Kaggle, the UCI Machine Learning Repository, and others.

For this project, we utilized the Crop_recommendation.csv dataset, which was obtained from Kaggle.

Once the dataset was successfully downloaded, we proceeded to perform an in-depth exploration and analysis using various data visualization and analytical techniques to better understand the structure and characteristics of the data.

Importing the libraries

IMPORTING THE LIBRARIES

In [5]:
```
import pandas as pd
import numpy as np
pd.set_option('max_colwidth', 20)
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 50)
import matplotlib.pyplot as plt
import scipy.stats as stats
import seaborn as sns
%matplotlib inline
plt.rcParams['figure.figsize'] = (12,8)
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
from ipywidgets import interact
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

Read the Dataset

The dataset can come in various formats such as .csv, .xlsx, .txt, or .json. In this project, the dataset is in .csv format.

We use the pandas library to load the dataset into a DataFrame using the read_csv() function. This function takes the path to the CSV file as its parameter and returns a structured DataFrame for further processing.

In [6]:
```
#Read the dataset
df = pd.read_csv(r"C:\Users\Asus\Desktop\OPTICROP\opticrop\Data\Crop_recommendation.csv")
df.head()
```

Out[6]:

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

Data Preprocessing

Before training a machine learning model, it is essential to preprocess the data to enhance its quality and relevance. Raw data often contains inconsistencies, noise, or unwanted patterns that may negatively affect model performance. Preprocessing helps transform the dataset into a clean and usable format.

This step involves the following common operations:

- Handling missing values
- Detecting and addressing outliers
- Managing categorical data

Handling Missing values

To begin, we analyze the structure and completeness of the dataset:

- df.shape is used to determine the number of rows and columns.
- df.info() provides data types and memory usage.
- df.isnull().sum() helps identify any missing values in each column.

After performing these checks, we observed that the dataset does not contain any null values. Therefore, we can safely skip the missing value treatment step.

DATA PREPROCESSING

```
In [7]: #Handling missing values

        df.shape

        df.info()

        df.isnull().sum()
```

```
Out[7]: (2200, 8)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   N            2200 non-null   int64
 1   P            2200 non-null   int64
 2   K            2200 non-null   int64
 3   temperature  2200 non-null   float64
 4   humidity     2200 non-null   float64
 5   ph           2200 non-null   float64
 6   rainfall     2200 non-null   float64
 7   label        2200 non-null   object
dtypes: float64(4), int64(3), object(1)
memory usage: 137.6+ KB
```

```
Out[7]: N              0
        P              0
        K              0
        temperature    0
        humidity       0
        ph             0
        rainfall       0
        label          0
        dtype: int64
```

```
In [8]: plt.figure(figsize=(8, 4))
        sns.boxplot(df)
```

Handling outliers

Outliers are extreme values that can skew the model's performance. To identify them, we utilize box plots, which provide a visual summary of the distribution and detect outliers effectively.
In our dataset, the Potassium (K) feature exhibited notable outliers, as shown in the boxplot generated using the Seaborn library.
To mathematically define outliers, we calculated the Interquartile Range (IQR) and determined the upper and lower bounds as follows:

- Upper Bound = Q3 + 1.5 × IQR
- Lower Bound = Q1 − 1.5 × IQR

To mitigate the effect of these outliers, a log transformation technique was applied. Additionally, we created a custom visualization function to display both the distribution and probability plots for the Potassium feature post-transformation.

Splitting the Dataset

Once preprocessing is complete, we divide the dataset into features (X) and target (y) variables. The target variable represents the label or output we aim to predict.

- X is constructed by dropping the target column from the DataFrame.
- y contains only the target labels.

We then use the train_test_split() function from the Scikit-learn library to split the data into training and testing sets.

Data Visualization and Analysis

Data visualization and analysis are essential components in understanding the structure and characteristics of a dataset. By converting raw data into visual formats—such as graphs, plots, and charts—we can simplify complex information and uncover hidden patterns, trends, and anomalies. This process not only enhances data interpretability but also supports more accurate and informed decision-making. Visualization tools help highlight relationships among features, identify correlations, and detect potential outliers or irregularities.

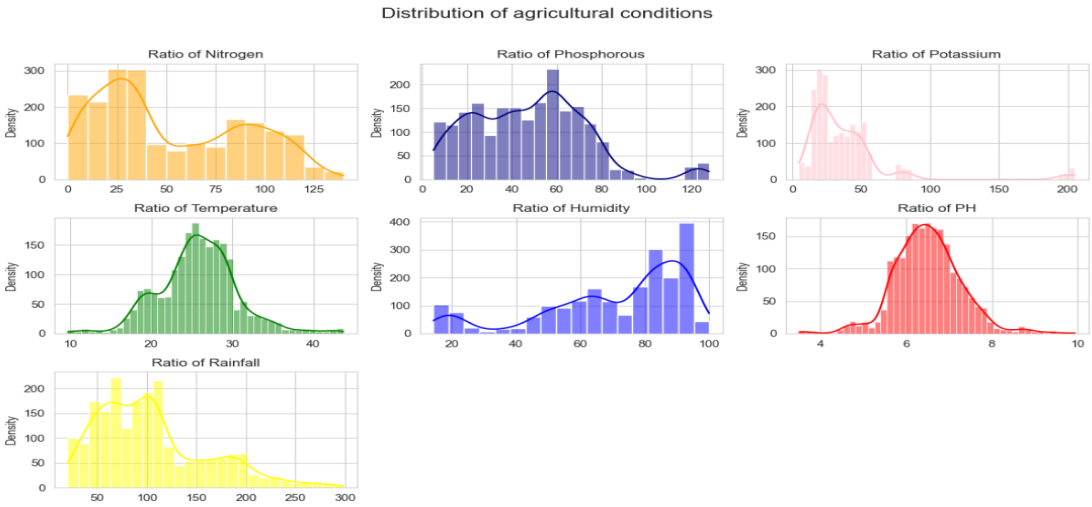
Combined with statistical analysis, visual exploration enables the extraction of meaningful insights that guide the development of effective models and strategies. Whether in scientific research, business intelligence, or machine learning, this step is vital for unlocking the full potential of data and ensuring a deeper, data-driven understanding.

Univariate Analysis

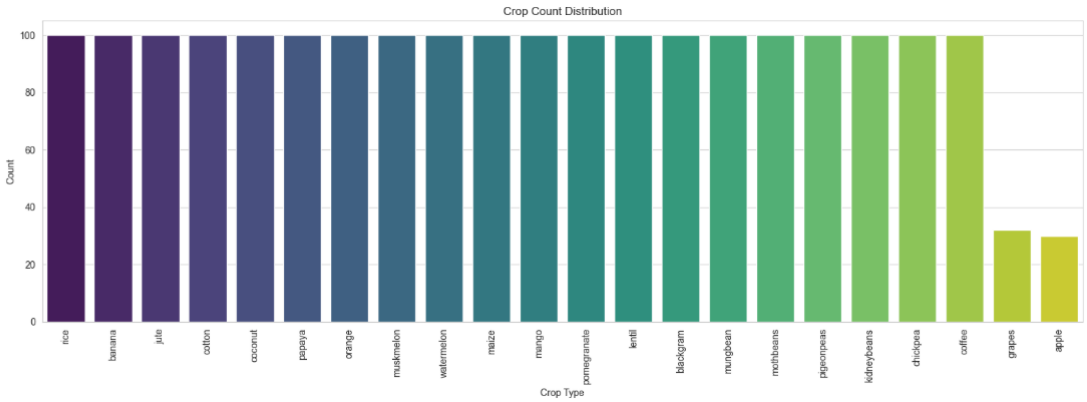
Univariate analysis involves examining and summarizing the distribution of individual features. This type of analysis helps in understanding the behavior of a single variable without considering the influence of others.

In this project, we used distplot and countplot from the Seaborn library:

- **distplot** provides a visual representation of the distribution of continuous numerical variables, highlighting data concentration and spread.
- **countplot** is used to display the frequency count of categorical variables.



In [18]:
```
#Countplot for Categorical Feature
# ================================
plt.figure(figsize=(16, 6))
sns.countplot(data=df, x='label', order=df['label'].value_counts().index, palette='viridis')
plt.xticks(rotation=90)
plt.title('Crop Count Distribution')
plt.xlabel('Crop Type')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```



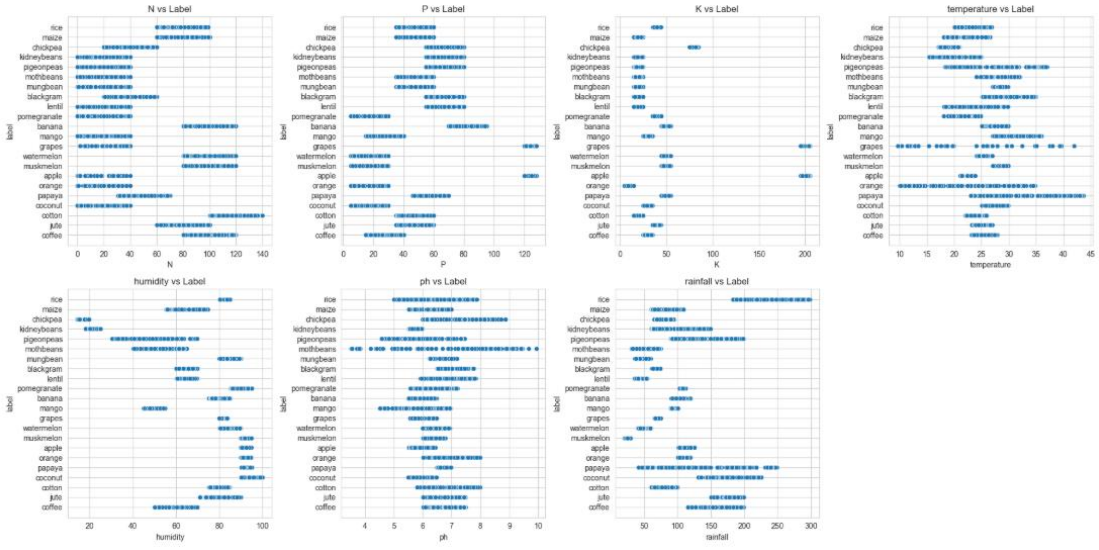
Bivariate Analysis

Bivariate analysis is used to explore the relationship between two variables. This helps determine whether changes in one variable are associated with changes in another.

In our analysis, we examined the relationship between humidity (a numerical feature) and label (the target variable) using suitable plots. This provided insight into how humidity may influence crop recommendations.

BIVARIATE ANALYSIS

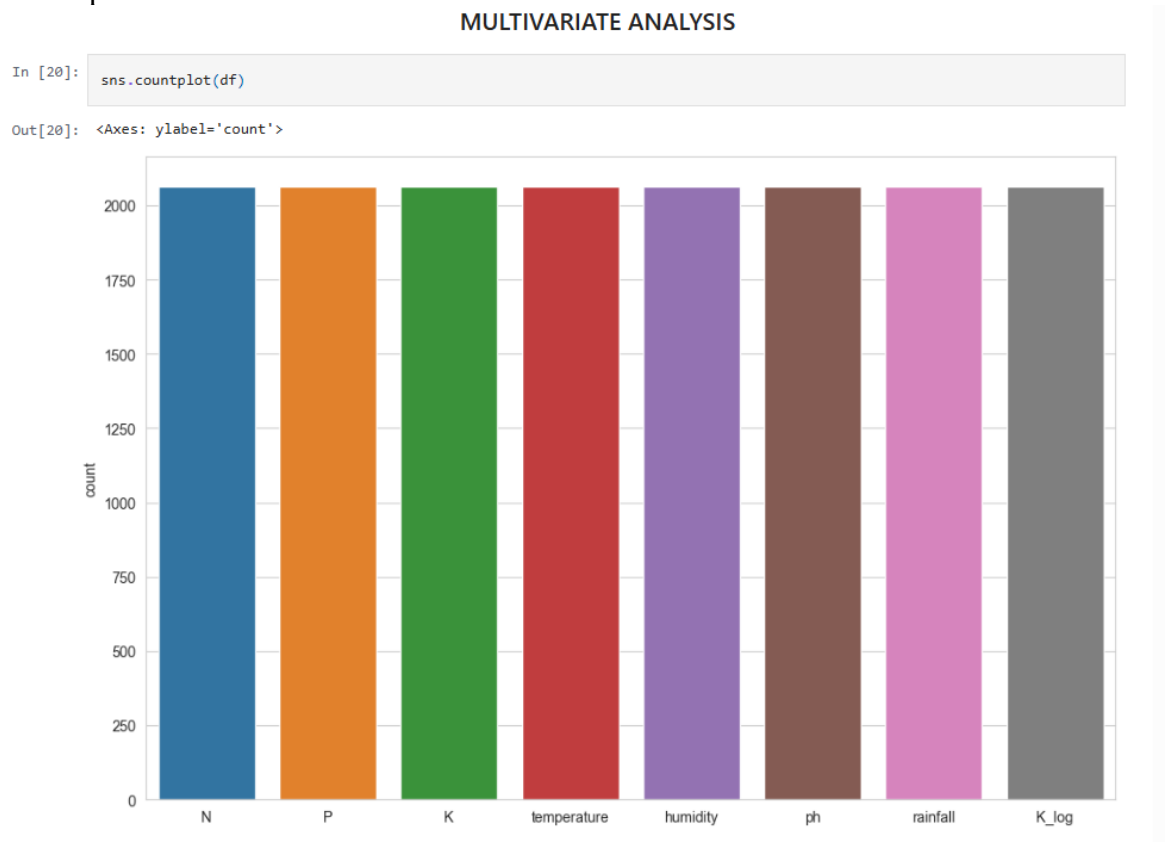
In [19]:
```
# Features to plot
features = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']

# Create subplots
plt.figure(figsize=(20, 10))
for i, feature in enumerate(features):
    plt.subplot(2, 4, i+1)
    sns.scatterplot(x=df[feature], y=df['label'])
    plt.title(f'{feature} vs Label')
    plt.xlabel(feature)
    plt.ylabel('label')

plt.tight_layout()
plt.show()
```



Multivariate Analysis

Multivariate analysis investigates the relationships among three or more variables simultaneously. This approach helps in understanding complex interactions within the dataset.

For this purpose, we used countplot to visualize how different categories of a target variable relate to multiple independent features. This allows us to assess the combined influence of multiple variables on the output.



Descriptive Statistical Analysis

Descriptive analysis involves summarizing the dataset's core characteristics using statistical methods. The describe() function in pandas offers a convenient summary:

- For **numerical features**, it provides metrics such as mean, standard deviation, minimum, maximum, and percentiles.
- For **categorical features**, it includes values like the most frequent entry, its count, and the number of unique entries.

This statistical overview is crucial for understanding the data's central tendencies and dispersion.

DESCRIPTIVE ANALYSIS

```
In [21]: df.describe()
```

Out[21]:

	N	P	K	temperature	humidity	ph	rainfall	K_log
count	2062.000000	2062.000000	2062.000000	2062.000000	2062.000000	2062.000000	2062.000000	2062.000000
mean	52.440349	47.706596	37.996605	25.770363	70.433297	6.502347	104.286486	3.462439
std	37.246143	25.479349	33.049629	4.885812	22.569963	0.785073	56.385635	0.587897
min	0.000000	5.000000	5.000000	9.724458	14.258040	3.504752	20.211267	1.791759
25%	22.000000	27.000000	20.000000	23.080864	58.469697	6.016597	62.940621	3.044522
50%	39.000000	48.000000	30.000000	25.768297	79.175605	6.469677	94.772563	3.433987
75%	87.000000	64.000000	46.000000	28.614586	89.130631	6.956328	132.787974	3.850148
max	140.000000	128.000000	205.000000	43.675493	99.981876	9.935091	298.560117	5.327876

Build Python code

Backend Development – Python with Flask

This section outlines the development of the Python backend using the Flask web framework, which serves as the connection between the trained machine learning model and the HTML user interface.

1. Importing Required Libraries

First, we import all the necessary Python libraries:

- Flask: to create the web server and handle routing.
- render_template: to load HTML pages.
- request: to handle form submissions from the frontend.
- joblib or pickle: to load the saved machine learning model.

Loading the Saved Model

The machine learning model, previously trained and saved using joblib or pickle, is loaded into memory to make predictions during user interaction.

Initializing the Flask App

An instance of the **Flask** class is created. This instance represents our web application.

Rendering HTML Pages

We define routing rules to connect specific URLs to HTML pages using the Flask @app.route decorator.

Handling Form Submission and Making Predictions

We create a route for handling form submission using a POST method. Input values are collected from the form, passed to the model, and the prediction result is returned to the user.

```python
import numpy as np
from flask import Flask, request, render_template
import pickle
import os

app = Flask(__name__)

# Absolute path to your model file
model_path =  os.path.join(os.path.dirname(__file__), 'model.pkl')

# Load the model safely
try:
    with open(model_path, 'rb') as file:
        model = pickle.load(file)
except Exception as e:
    raise Exception(f"Could not load the model: {e}")

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/FindYourCrop', methods=['GET', 'POST'])
@app.route('/FindYourCrop', methods=['GET', 'POST'])
def FindYourCrop():
    if request.method == 'POST':
        try:
            # Match input names from HTML
            N = float(request.form['N'])
            P = float(request.form['P'])
            K = float(request.form['K'])
            temperature = float(request.form['temperature'])
            humidity = float(request.form['humidity'])
            ph = float(request.form['ph'])
            rainfall = float(request.form['rainfall'])

            # Log transformation
            K_log = np.log(K + 1)

            # Prediction
            features = np.array([[N, P, K_log, temperature, humidity, ph, rainfall]])
            prediction = model.predict(features)

            return render_template('FindYourCrop.html', prediction=prediction[0])

            # Log transformation
            K_log = np.log(K + 1)

            # Prediction
            features = np.array([[N, P, K_log, temperature, humidity, ph, rainfall]])
            prediction = model.predict(features)

            return render_template('FindYourCrop.html', prediction=prediction[0])

        except Exception as e:
            return render_template('FindYourCrop.html', prediction=f"⚠️ Error: {e}")

    return render_template('FindYourCrop.html')

if __name__ == "__main__":
    app.run(debug=True)
```

Running the Application

To run the Opticrop web application, follow these deployment steps using **Visual Studio Code**:

1. **Launch the Project:**
    - Open **Visual Studio Code**.
    - Import or open the project directory that contains all necessary files and folders, including HTML templates, the trained model, and the main application script (app.py).
2. **Start the Flask Server:**
    - Run the app.py file by executing it in the integrated terminal.
    - Once the Flask server starts, a local server URL (typically http://127.0.0.1:5000/) will be displayed in the terminal.
    - Click or paste this URL into your web browser to launch the application.
3. **Navigating the Application:**
    - Upon accessing the URL, the application will load the **Home Page (home.html)**, which serves as the main landing interface.
    - From the top navigation menu, clicking on the **"About"** link will redirect you to the **About Page (about.html)**, where details about the project and its objectives are displayed.
    - Selecting the **"Find Your Crop"** option from the navigation bar opens the **Find Crop Page (findyourcrop.html)**. Here, users can enter values related to soil nutrients, temperature, humidity, pH, and rainfall.
4. **Generating a Prediction:**
    - After filling in the required values on the "Find Your Crop" form, clicking the **"Predict"** button will submit the data to the backend.
    - The trained machine learning model processes the input and returns a crop recommendation.
    - The predicted result is then displayed on the same page, providing users with real-time, actionable insights.

This marks the successful execution of the Opticrop application, enabling end-to-end interaction from user input to intelligent prediction via a seamless web interface.

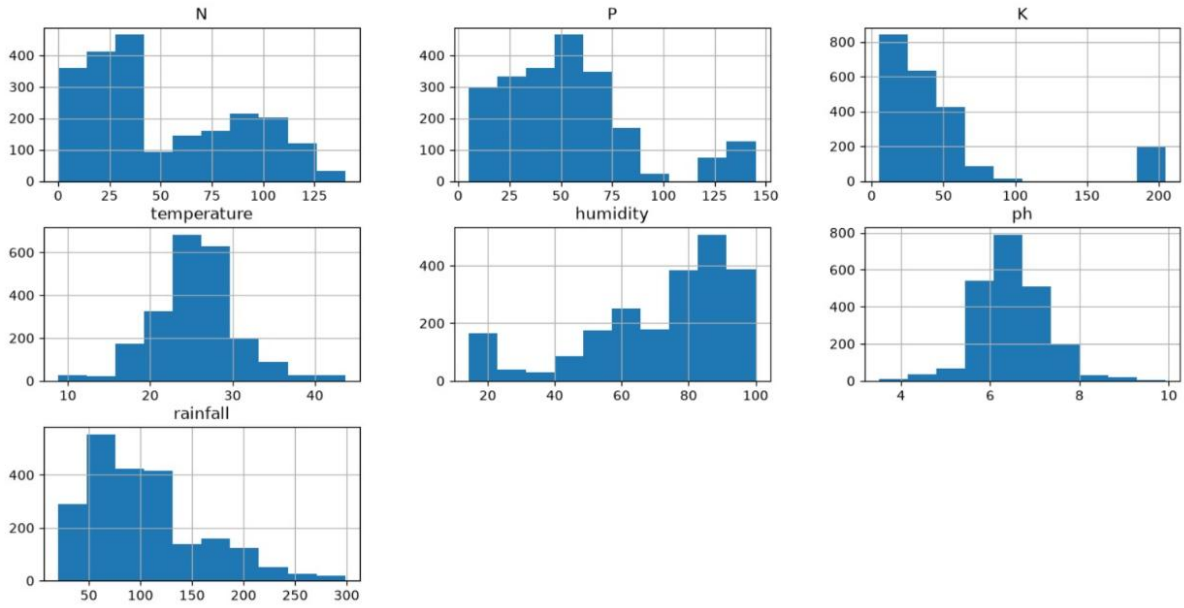
1. Data Visualization and Analysis Section

After "Univariate Analysis", add the Histogram image.

Figure 1: Feature Distribution Histograms

Description:

The histogram plots illustrate the distribution of important agricultural parameters including Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall. These visualizations help identify the spread, skewness, and concentration of data values, providing insights into feature behavior and assisting in preprocessing decisions.



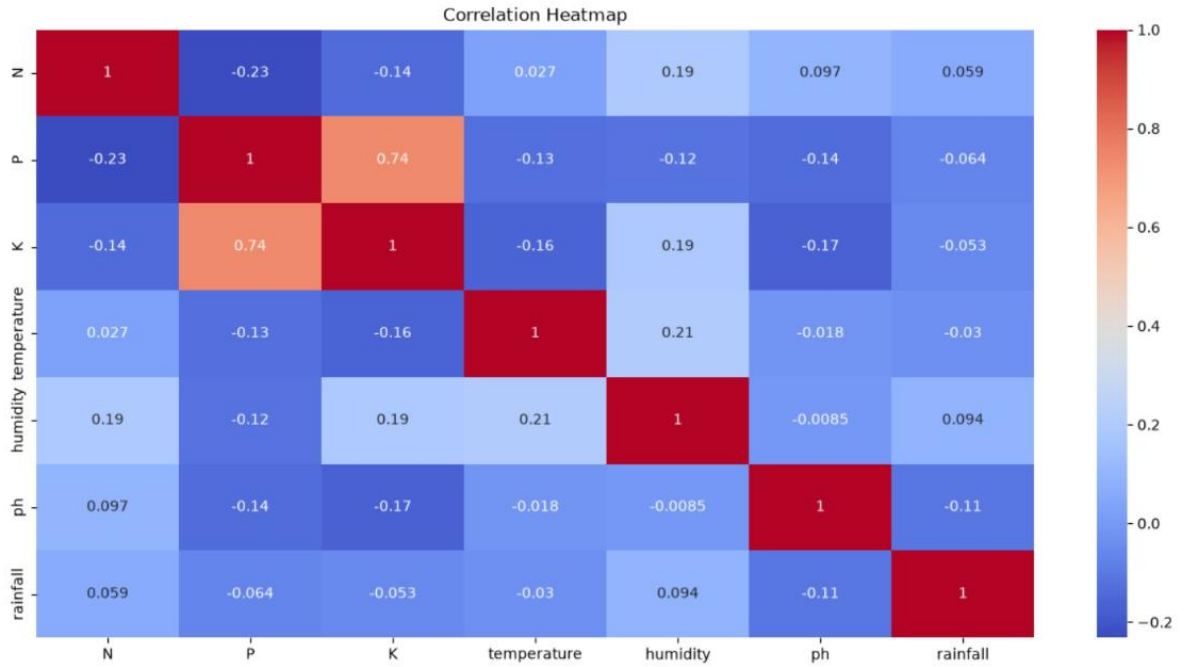
2. Correlation Analysis Section

After "Multivariate Analysis", add the Heatmap image.

Figure 2: Correlation Heatmap of Agricultural Features

Description:

The correlation heatmap represents the relationships among the agricultural features used in crop prediction. Positive correlations indicate features that increase together, while negative correlations indicate inverse relationships. The heatmap helps identify dependencies among variables and supports feature selection during model development.

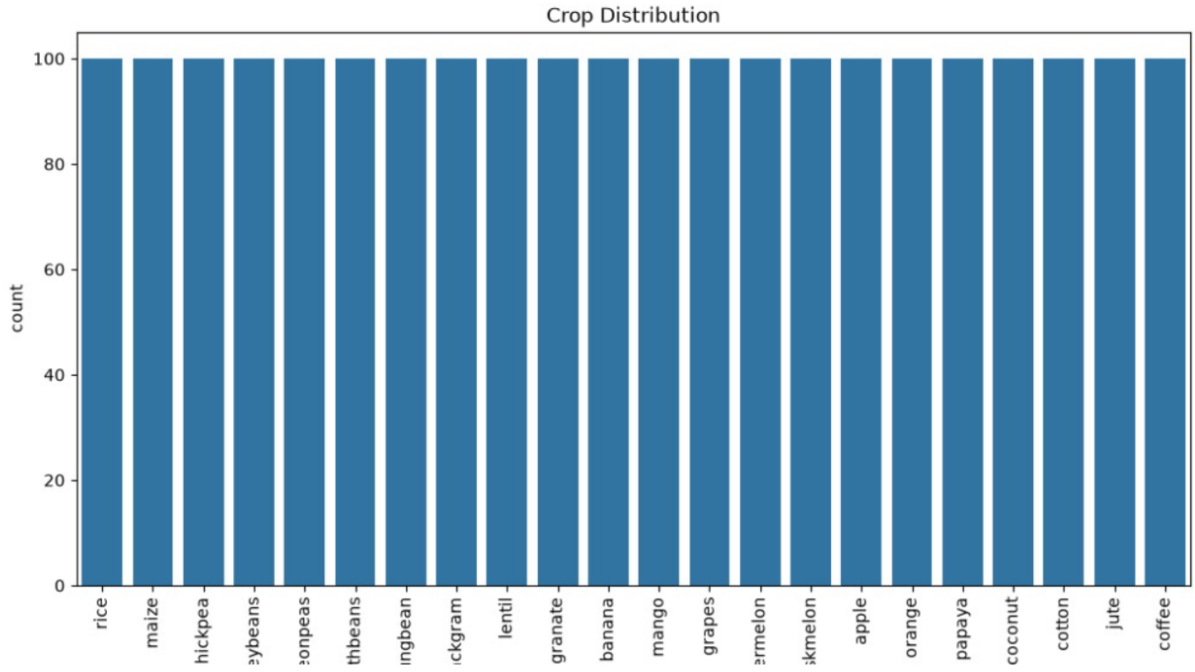


3. Crop Distribution Analysis

Add the Crop Distribution bar chart after descriptive analysis.

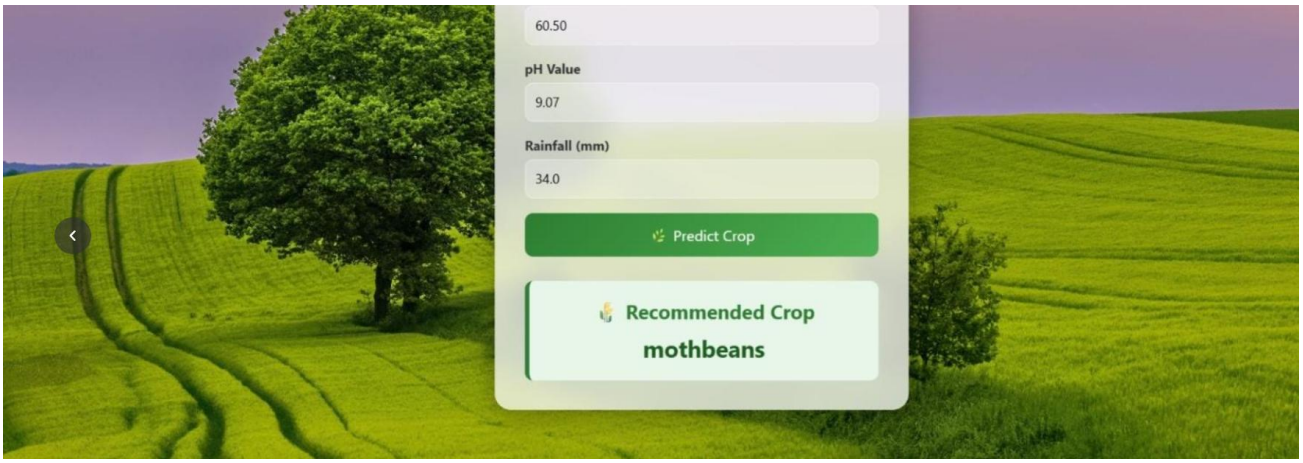
Figure 3: Crop Distribution Across Dataset

Description:
The crop distribution graph shows the frequency of each crop class present in the dataset. The balanced distribution of crop categories ensures that the machine learning model is trained without significant class imbalance, improving prediction accuracy and reliability.



4. Application Building Section
Under "Building HTML Pages" or "Find Your Crop Page".
Figure 4: OptiCrop User Interface
Description:
The OptiCrop web application provides an intuitive interface that allows users to input soil nutrient values (N, P, K), environmental conditions such as temperature, humidity, pH, and rainfall. Based on these parameters, the machine learning model predicts and recommends the most suitable crop for cultivation.

5. Prediction Result Section
Recommended Crop: mothbeans
Figure 5: Crop Prediction Result
Description:
The trained machine learning model processes the user-provided agricultural parameters and generates the optimal crop recommendation. In this example, the model recommends mothbeans as the most suitable crop under the given environmental conditions.



Exploring the Website Pages

Home Page

The home page introduces the OptiCrop system and provides navigation options for crop recommendation and agricultural analysis.

Crop Recommendation Page

Users enter Nitrogen, Phosphorous, Potassium, temperature, humidity, pH, and rainfall values. The system predicts the most suitable crop.

Crop Suitability Assessment Page

Users select a crop and enter environmental parameters. The system evaluates crop compatibility and productivity potential.

Research and Analysis Page

Researchers can analyze crop-environment relationships using graphical visualizations and statistical reports.

Conclusion:

OptiCrop successfully demonstrates the application of machine learning in agriculture by providing intelligent crop recommendations based on environmental and soil parameters. The system supports precision farming, improves resource efficiency, and contributes to sustainable agricultural practices. Future enhancements may include IoT integration, weather forecasting APIs, disease prediction systems, and real-time agricultural monitoring capabilities.